

Lab 4 – dplyr

Contents

Introduction	1
Logical Operators	2
Equality Logic	3
Compound Statements and Negation	4
Value Matching with %in%	4
dplyr verbs	5
filter	5
mutate	6
summarize	10
group_by	10
Problem Sets	13

This lab serves as our introduction to the **dplyr** (pronounced dee-ply-R) package in **R** which provides a cohesive framework for working with and manipulating data. Let's begin by creating a new R code block and copying the code down below. In particular, make note of the fact that we now need to include `library(dplyr)`, along with `library(ggplot2)`, at the top of our documents when we are ready to knit.

```
## Copy and paste the code chunk below into the top of your R Markdown document
library(ggplot2)
library(dplyr)

theme_set(theme_bw(base_size = 14))

## College data
college <- read.csv("https://collinn.github.io/data/college2019.csv")

## For this lab, we don't need all of the columns so we will
# select only a few of them
college <- select(college, Name, State, Enrollment, Type,
                  Region, Adm_Rate, ACT_median, Cost, Net_Tuition,
                  Avg_Fac_Salary, Debt_median)

## Load data from ggplot package
data(mpg)
```

Introduction

Data manipulation can mean a lot of things. For this lab, we will focus on the four main operations we will be using on standard, table-shaped datasets. These operations are organized around verbs which will correspond to functions within the **dplyr** package. The verbs for today are:

1. **Filter** – Filtering involves creating subsets of our data according to some criterion. For example, we

may be interested in filtering our college dataset to only include private college or colleges located in Iowa

2. **Mutate** – Mutating is involved with the creation of new variables in our dataset or the transformation of existing one
3. **Summarize** – Summarizing is used to calculate descriptive statistics
4. **Grouping** – Grouping allows us to specify groups of our data for the preceding functions rather than acting upon the entire dataset, i.e., find the mean value of these groups rather than of all of our observations together

The `dplyr` package contains a suite of functions that accommodate these procedures, often *chained together* in a sequence of operations, connected with our pipe operator, `%>%` (Ctrl+Shift+M for PC, Cmd+Shift+M for Mac). Recall that the pipe operator takes the left hand side and passes it into the function on the right hand side. This creates a “pipe” where I can string together a sequence of operations.

For example, if I wanted to find the average cost for public and private schools in Iowa, my code may look something like this:

```
college %>% filter(State %in% "IA") %>% # Filter by state to keep only IA
  group_by(Type) %>% # Group by type of school
  summarize(meanCost = mean(Cost)) # Summarize the average cost
```

```
## # A tibble: 2 x 2
##   Type      meanCost
##   <chr>      <dbl>
## 1 Private    46645.
## 2 Public     21420
```

Here, we first *filter* our data to include only schools in Iowa, then we *group by* the `Type` variable so that when I *summarize* my data by finding the mean, I am doing so within each of my defined groups.

Along with some helper functions to facilitate these operations, the main functions we will be using from the `dplyr` package are as follows:

Verb/Function	Meaning
<code>filter</code>	pick specific observations (i.e. specific rows)
<code>mutate</code>	add new columns to a data frame
<code>summarize</code>	aggregate many rows into a summary measure
<code>group_by</code>	perform subsequent operations separately on the groups created by a categorical variable

Often when organizing or manipulating our data, we are doing so according to a set of conditions (i.e., grab only private schools or schools with enrollment larger than 2000 students). Rather than try to find and grab these observations directly, we instead specify these conditions as *logical* statements for R to process – as such, it doesn’t matter how large our dataset is or if the rows get shuffled around; so long as we correctly specify a set of logical conditions, we will always end up with our intended output. This need motivates our next section, a short exploration of the world of *logical operators*.

Logical Operators

Before beginning with `dplyr`, it will be helpful to introduce the idea of *logical operators*. A few common logical operators are described in the table below:

Operator	Description
<code>==</code>	equal to

Operator	Description
<code>!=</code>	<i>not</i> equal to
<code>%in%</code>	equal to <i>any element</i> in a given vector
<code><</code> and <code><=</code>	less than (strict) or less than or equal to
<code>></code> and <code>>=</code>	greater than (strict) or greater than or equal to
<code>&</code>	and
<code> </code>	or
<code>!</code>	not

Though this seems like a lot, they don't really have to be memorized; many of them will soon become intuitive and we will have many opportunities to practice.

Equality Logic

Briefly, a logical operator is an operator (like `+`) that evaluates if a particular statement is **TRUE** or **FALSE** and returns those logical values accordingly. Consider a simple case using single scalar values

```
## Using less than operator
```

```
3 < 4
```

```
## [1] TRUE
```

```
## Equality operator
```

```
3 == 4
```

```
## [1] FALSE
```

A peculiar aspect of R (and most programming languages) is that a single `=` is used for *assignment* (as in, `x = 4` will assign the value 4 to the variable `x`) rather than checking equality. Typing this into your R console will result in an error.

```
3 = 4
```

```
## Error in `3 = 4`:
```

```
## ! invalid (do_set) left-hand side to assignment
```

A common use of logical operators is to evaluate a *vector* against some condition, determining for which elements of a vector a logical statement is true or false. Consider a vector of length five, using logical operators to determine which values are greater than or equal to 3

```
## Create a vector with values 1,2,3,4,5
```

```
x <- 1:5
```

```
x
```

```
## [1] 1 2 3 4 5
```

```
## Determine for which positions the value in the vector is greater than
```

```
## or equal to 3
```

```
x >= 3
```

```
## [1] FALSE FALSE TRUE TRUE TRUE
```

In this case, the operation returns a logical vector indicating that our statement is **TRUE** in the third, fourth, and fifth positions and **FALSE** everywhere else. Generally speaking, the length of the logical vector returned will be the same length as the vector on the left hand side.

A very common use of logical vectors is *subsetting*, whereby we retain only those elements of a vector for which a condition is true. We can do this by either assigning our logical values to a new vector, or by using them directly inside of the brackets, `[]`:

```
## Create a vector of logical values where x >= 3
## And only keep those locations where it is true
x[x >= 3]
```

```
## [1] 3 4 5
```

Compound Statements and Negation

There are two important ways in which we might want to modify our logical vectors. The first is to *negate* them, which involves swapping all of the TRUE/FALSE values. We do this with an exclamation point !

```
x <- 1:5
```

```
## Where is x less than 4
x < 4
```

```
## [1] TRUE TRUE TRUE FALSE FALSE
```

```
## Where is x not less than 4
!(x < 4)
```

```
## [1] FALSE FALSE FALSE TRUE TRUE
```

Typically, we negate logical vectors with a combination of ! and ()

Sometimes we need to express logical statements that are more complicated than a single logical check. For example, consider a vector with the numbers 1 through 10 where we want to take all of the numbers between 3 and 7. Typing this directly will result in an error:

```
x <- 1:10
3 < x < 7
```

```
## Error in parse(text = input): <text>:2:7: unexpected '<'
## 1: x <- 1:10
## 2: 3 < x <
##      ^
```

What we need instead is to combine logical statements with a compound operator like &:

```
x <- 1:10
(3 < x) & (x < 7) # x greater than 3 AND x less than 7
```

```
## [1] FALSE FALSE FALSE TRUE TRUE TRUE FALSE FALSE FALSE
```

```
## We can use this to subset
x[(3 < x) & (x < 7)]
```

```
## [1] 4 5 6
```

Value Matching with %in%

The last operator worth exploring in some more detail is the the value matching operator, %in%. The value operator always asks the question: “what values on my *left hand side* (LHS) are also included on the *right hand side* (RHS). This distinction is important in specifying the order that we include things:

```
## We see that the first and second position of LHS are also in RHS
c("a", "a", "b", "c") %in% c("a", "d", "r")
```

```
## [1] TRUE TRUE FALSE FALSE
```

```
## We see the first position on LHS is also in RHS
c("a", "d", "r") %in% c("a", "a", "b", "c")
```

```
## [1] TRUE FALSE FALSE
```

As we will shortly see, keeping this straight will be easier than you might think. One final thing to keep in mind when value matching is that we should never use `==`, as it will check position-by-position for equality; this is a common source of error for new R users.

```
x <- c("kitty", "cat")
y <- c("cat", "kitty")

## Because positions don't align, these will both be false
x == y
```

```
## [1] FALSE FALSE
```

```
## Because %in% checks matching, this will be true as each word in x
## is also in y
x %in% y
```

```
## [1] TRUE TRUE
```

dplyr verbs

filter

We now move on to investigating the `dplyr` functions. Each of the functions we will be working with today takes as its first argument a data frame. Additionally, each of the functions *returns* a data frame. Because everything will involve a data frame coming in and a data frame going out, we will make use of the pipe operator to chain our operations together. As the sequence of our operations becomes more sophisticated, we will find that using the pipe makes the entire sequence much easier to read and ultimately understand.

The first function we will cover is `filter()`, where we will now have a chance to make use of our logical operators from the previous section. `filter()` works by subsetting our data according to which observations (rows) satisfy a logical condition. For example, if I wanted to filter my entire `college` dataset to contain *only* schools with a median ACT greater than 29, I can do so with a logical operator like this:

```
## I need to save my operations to a new variable if I want to use it
act30 <- college %>% filter(ACT_median > 29)
```

```
## See how the dimensions have changed
dim(college)
```

```
## [1] 1095 11
```

```
dim(act30)
```

```
## [1] 85 11
```

```
## The head() function allows me to see the first 6 rows of my data
head(act30)
```

##	Name	State	Enrollment	Type	Region	Adm_Rate
## 1	Amherst College	MA	1855	Private	New England	0.1281
## 2	Bentley University	MA	4177	Private	New England	0.4321
## 3	Binghamton University	NY	13990	Public	Mid East	0.3981
## 4	Boston College	MA	9639	Private	New England	0.2789
## 5	Boston University	MA	17238	Private	New England	0.2209
## 6	Brandeis University	MA	3627	Private	New England	0.3115
##	ACT_median	Cost	Net_Tuition	Avg_Fac_Salary	Debt_median	
## 1	33	71300	27541	116775	11900	

```
## 2      30 66180      30307      126936      23250
## 3      30 25846       8187       88011      16750
## 4      33 70588      34327      137907      16000
## 5      32 70216      33686      127224      22500
## 6      31 70848      24212      118584      24708
```

The `filter` function can take more than one expression if needed, either separated by commas or the logical operator `&`. Here, we illustrate filtering our dataset to include only those with median ACT of 30 or more *and* colleges located in Iowa (IA) or Minnesota (MN)

```
## REMEMBER: I need to assign this to a new variable name to use it again
act30_iamn <- college %>% filter(ACT_median > 29,
                                State %in% c("IA", "MN"))
head(act30_iamn)
```

```
##           Name State Enrollment   Type Region Adm_Rate ACT_median  Cost
## 1  Grinnell College   IA      1683 Private Plains   0.2438      32 65814
## 2 Macalester College  MN      2160 Private Plains   0.4124      31 66280
##   Net_Tuition Avg_Fac_Salary Debt_median
## 1      20369      101979      15000
## 2      23225      95607      19432
```

Notice how the `%in%` operator is used here: `State %in% c("IA", "MN")` first considers the variable `State` and then matches it to each of the values in the vector `c("IA", "MN")`

Problem 1: Use `filter()` to create a dataset containing only private colleges with an acceptance rate below 50%. How many colleges are left?

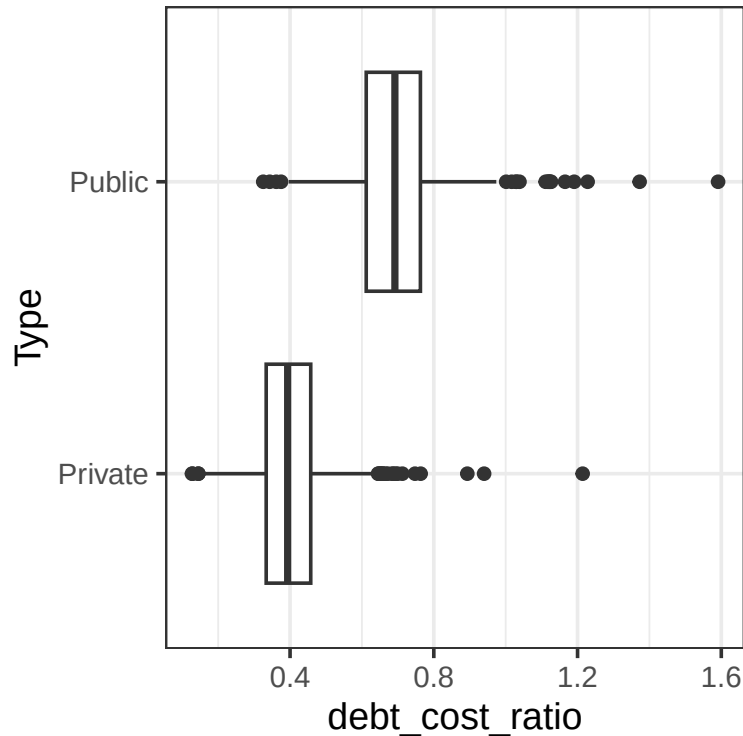
mutate

The purpose of mutating is typically to create new variables out of old ones. For example, our college dataset contains variables for both cost and median debt, and it may be of interest to see for a given school what the typical ratio of debt to cost is

```
## Use mutate to create new variable called debt_cost_ratio
debt_cost <- college %>%
  mutate(debt_cost_ratio = Debt_median/Cost)
```

Within the `mutate` function, we see `debt_cost_ratio = Debt_median/Cost`; the left hand side of this indicates the new variable name that we are creating, while the right hand side shows us how we created it out of the existing variables `Debt_median` and `Cost`. We are often creating new variables with the intent of analyzing them, which we can do easily with `ggplot`:

```
## Notice I am using debt_cost data.frame since it contains
## the new variable that I created
ggplot(debt_cost, aes(debt_cost_ratio, Type)) +
  geom_boxplot()
```



From this, we see that students at public colleges take a generally much greater proportion of the total cost of college out as debt.

Problem 2: Use `mutate()` to create a variable called `cost_to_debt_ratio` equal to `Cost / Debt_median`. Create a plot to compare this against `Adm_Rate`

Using the `ifelse()` function

A very common trick in data analysis, and one that we have seen a few times in class, is that of changing a continuous variable into a categorical one. A very useful function for doing so is the `ifelse()` function. `ifelse()` takes three arguments:

1. A logical expression
2. What to return if TRUE
3. What to return if FALSE

For example, consider a variable `x` with the values 1-10 which we may want to assign to one of two groups, depending on its value.

```
x <- 1:10
x

## [1] 1 2 3 4 5 6 7 8 9 10
# If x < 5, return "little", otherwise return "big"
ifelse(x < 5, "little", "big")

## [1] "little" "little" "little" "little" "big"    "big"    "big"    "big"
## [9] "big"    "big"
```

This is saying, “Everywhere where `x` is less than 5, return the value ‘little’, otherwise, return the value ‘big’.”

Now consider our college dataset with the variable `Adm_Rate`, indicating the percentage of applicants that are admitted. This is a continuous variable by default, but we could use it to create a *new* variable indicating

whether or not a school is considered *selective*:

```
ia_select <- college %>%
  filter(State %in% "IA") %>%
  mutate(Selective = ifelse(Adm_Rate < 0.3, "selective", "not selective"))

# How many selective schools are in Iowa?
with(ia_select, table(Selective))

## Selective
## not selective      selective
##           24           1
```

Using the `case_when()` function

Note to STA-209 students: this section addressing the `case_when()` function is a generalization of the `ifelse()` function above, extended to more than two categories with slightly more cumbersome syntax. While `ifelse()` will be far more commonly used in this course, some students have asked for a short section on `case_when()` which is included for completeness here.

Frequently in data analysis we wish to change the *values* of one variable into something else; this could be as simple as changing values of a categorical variable (e.g., “Jan” to “January”) or changing a continuous variable into a categorical one. Both cases are handled with the `case_when()` function. `case_when()` works by creating a comma-separated list of logical statements and the value that should be adopted if the logical statement is true.

For example, consider a variable `x` with the values 1-10 which we may want to assign to one of two groups, depending on its value:

(Note: for the next few examples, I use the function `cbind()` or “column bind” so that we can see the original vector alongside the values returned by `case_when()`)

```
x <- 1:10
x

## [1] 1 2 3 4 5 6 7 8 9 10
## If x < 5, return "little", otherwise return "big"
cbind(x, case_when(
  x < 5 ~ "little",
  x >= 5 ~ "big"
))

##           x
## [1,] "1"  "little"
## [2,] "2"  "little"
## [3,] "3"  "little"
## [4,] "4"  "little"
## [5,] "5"  "big"
## [6,] "6"  "big"
## [7,] "7"  "big"
## [8,] "8"  "big"
## [9,] "9"  "big"
## [10,] "10" "big"
```

Note that each line has the form `condition ~ value`, where the LHS is a condition to be evaluated and, if true, will return the specified value.

Unlike `ifelse()` which you may have seen in other classes, `case_when()` can handle any number of cases.

However, if individual observations do not match any of the conditions (such as the case for March in the example below), the function will return a value of NA

```
## List of months
month.abb

## [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"

## Specify temperature of the month
cbind(month.abb, case_when(
  month.abb %in% c("Dec", "Jan", "Feb") ~ "cold",
  month.abb %in% c("Jun", "Jul", "Aug") ~ "hot"
))

##      month.abb
## [1,] "Jan"     "cold"
## [2,] "Feb"     "cold"
## [3,] "Mar"     NA
## [4,] "Apr"     NA
## [5,] "May"     NA
## [6,] "Jun"     "hot"
## [7,] "Jul"     "hot"
## [8,] "Aug"     "hot"
## [9,] "Sep"     NA
## [10,] "Oct"    NA
## [11,] "Nov"    NA
## [12,] "Dec"    "cold"
```

We can use the `.default` argument to specify what value to take if none of the conditions match. Note, however, that the argument starts with a `.` and uses `=` instead of `~`

```
## Specify temperature of the month
cbind(month.abb, case_when(
  month.abb %in% c("Dec", "Jan", "Feb") ~ "cold",
  month.abb %in% c("Jun", "Jul", "Aug") ~ "hot",
  .default = "other"
))

##      month.abb
## [1,] "Jan"     "cold"
## [2,] "Feb"     "cold"
## [3,] "Mar"     "other"
## [4,] "Apr"     "other"
## [5,] "May"     "other"
## [6,] "Jun"     "hot"
## [7,] "Jul"     "hot"
## [8,] "Aug"     "hot"
## [9,] "Sep"     "other"
## [10,] "Oct"    "other"
## [11,] "Nov"    "other"
## [12,] "Dec"    "cold"
```

This is especially useful if we wish to create a categorical variable out of a continuous one, based on ranges of values

```
ia_select <- college %>%
  filter(State %in% "IA") %>%
  mutate(Selective = case_when(
```

```

    Adm_Rate < 0.3 ~ "Selective",
    .default = "Not Selective"
  ))

# How many selective schools are in Iowa?
with(ia_select, table(Selective))

```

```

## Selective
## Not Selective      Selective
##           24           1

```

summarize

The `summarize()` function helps us calculate descriptive statistics requiring an aggregation of rows. As a result, we will nearly always end up with fewer rows than with which we begin. For example, I may be interested in a table of quantitative statistics describing the variable `Cost`:

```

## Notice here that I am NOT assigning it to a new variable name.
## That will cause my summary to print out, but it will not save it in my
## environment for later use. If you ever cannot find your modified data.frame
## make sure you saved it to a new variable first
college %>% summarize(meanCost = mean(Cost),
                      sdCost = sd(Cost),
                      minCost = min(Cost),
                      maxCost = max(Cost))

```

```

##   meanCost sdCost minCost maxCost
## 1    37129 15251  12223  75735

```

The general syntax for something like `meanCost = mean(Cost)` is such that the name of our new variable is on the left hand side, while the function of our original variable `Cost` is on the right hand side. You will also note that the data frame returned contains *only* the new variables that were created in the `summarize()` function.

While the `summarize()` function is useful, it is most frequently used with `group_by()`, which we illustrate next.

Problem 3: Use `summarize()` to find the mean and median enrollment of colleges in the dataset

group_by

Typically when we are summarizing information in our data frame, we want to do so at a *group level*. We can indicate that we wish to do this using the `group_by()` function. The function works in two steps:

1. First, `group_by()` determines which categorical variables we wish to use to create groups and creates invisible tags so that other functions know these groups exist. Typically this is one variable, but it can also be several.
2. Whenever functions from the `dplyr` package see data that is grouped (typically with the `summarize()` function), it performs whatever operations it intends to at the group level.

The application of this is straightforward enough. Here, we indicate that we wish to group our college dataset by “Region” and then find the mean cost of attending

```

college %>% group_by(Region) %>%
  summarize(meanCost = mean(Cost))

```

```

## # A tibble: 8 x 2
##   Region      meanCost

```

```
##   <chr>           <dbl>
## 1 Far West       40409.
## 2 Great Lakes    37403.
## 3 Mid East       43212.
## 4 New England    45682.
## 5 Plains         35810.
## 6 Rocky Mountains 26503.
## 7 South East     32933.
## 8 South West     31289.
```

All of the functions that we have introduced in this lab form a cohesive framework for interactive data analysis. In particular, they were designed with the philosophy in mind that they should be able to be chained together to succinctly perform some of the most common applications. Consider for example the code below where we

1. First limit our colleges to those located in Iowa (IA), Missouri (MO), and Minnesota (MN) using `filter()`
2. We then compute the debt to cost ratio for each school just as we did before using `mutate()`
3. We indicate that we wish to group our data according to *both* State and Type, which we do with `group_by()`
4. Finally, we summarize our data to find the mean and standard deviation of the debt to cost ratio using `summarize()`

```
college %>%
  filter(State %in% c("IA", "MO", "MN")) %>%
  mutate(debt_cost_ratio = Debt_median / Cost) %>%
  group_by(State, Type) %>%
  summarize(mean_dcr = mean(debt_cost_ratio),
            sd_dcr = sd(debt_cost_ratio))
```

```
## # A tibble: 6 x 4
## # Groups:   State [3]
##   State Type    mean_dcr sd_dcr
##   <chr> <chr>      <dbl> <dbl>
## 1 IA    Private    0.399 0.0597
## 2 IA    Public     0.780 0.0967
## 3 MN    Private    0.423 0.0783
## 4 MN    Public     0.673 0.0797
## 5 MO    Private    0.420 0.121
## 6 MO    Public     0.716 0.0459
```

Very quickly we are able to see that public schools in Iowa have the largest debt to cost ratio, while private schools in Iowa have the lowest.

Problem 4: Find the mean acceptance rate (`Adm_Rate`) for public and private colleges. Which type has the higher average acceptance rate?

Mutate vs Summarize when grouping

It is worth noting an important distinction between `mutate()` and `summarize()` when using `group_by()`, namely:

- `mutate()` will always create a new variable with the same number of rows
- `summarize()` will always reduce the number of rows to the number of groups. It will also *delete* any variables that are not a part of the grouping or the summarizing

Consider this toy example to see how the behavior changes between each. We have a `group` column, specifying group, and a `val` column, specifying a value. This dataset is constructed so that group “A” has a mean value of 5, while group B has a mean value of 50.

```
df <- data.frame(group = rep(c("A", "B"), each = 4),
                 val = c(rnorm(4, 5), rnorm(4, 50)))
df
```

```
##   group    val
## 1     A 6.2193
## 2     A 5.0094
## 3     A 4.2761
## 4     A 4.2685
## 5     B 50.0128
## 6     B 49.3216
## 7     B 49.5927
## 8     B 51.4156
```

If I first group and then mutate, it will add an additional column, returning the mean value for each group

```
df %>% group_by(group) %>%
  mutate(meanVal = mean(val))
```

```
## # A tibble: 8 x 3
## # Groups:   group [2]
##   group    val meanVal
##   <chr> <dbl>   <dbl>
## 1 A      6.22    4.94
## 2 A      5.01    4.94
## 3 A      4.28    4.94
## 4 A      4.27    4.94
## 5 B     50.0    50.1
## 6 B     49.3    50.1
## 7 B     49.6    50.1
## 8 B     51.4    50.1
```

Contrast this with the summarize function

```
df %>% group_by(group) %>%
  summarize(meanVal = mean(val))
```

```
## # A tibble: 2 x 2
##   group meanVal
##   <chr>   <dbl>
## 1 A      4.94
## 2 B     50.1
```

When summarizing, note that all of the columns not used for either grouping or summarizing have been removed.

Finally, it is useful to be aware of the helper function `n()` which can be used inside of `summarize` – this permits us to display the number of observations in our groups in addition to other statistical summaries.

```
college %>%
  filter(State %in% c("IA", "MO", "MN")) %>%
  mutate(debt_cost_ratio = Debt_median / Cost) %>%
  group_by(State, Type) %>%
  summarize(Count = n()) # helper function
```

```
## # A tibble: 6 x 3
## # Groups:   State [3]
```

```
##   State Type   Count
##   <chr> <chr>  <int>
## 1 IA     Private   22
## 2 IA     Public     3
## 3 MN     Private   16
## 4 MN     Public    10
## 5 MO     Private   18
## 6 MO     Public    10
```

Problem Sets

Problem 5: Filter the college dataset to include only schools located in the Plains and the Great Lakes regions and with enrollments less than 20,000. Using your filtered data, create a two-way table using `table()` with the variables `Region` and `Type`. Based on this table, in which region do most private schools tend to be located?

Problem 6: Using the college dataset, create a new variable called `Size` that takes the value "large" if `Enrollment` is greater than or equal to 10000 and "small" if `Enrollment` is less than 10000. Then create an appropriate bar chart to determine which region has the greatest proportion of small schools.

Problem 7: Using the college dataset, create a new variable called `Size` that takes the values "small" when enrollment is less than 5000, "medium" when enrollment is 5,000 or greater but less than 10,000, and "large" otherwise. Then create an appropriate bar chart to determine which region has the greatest proportion of small schools.

Problem 8: Using the tips dataset, do the following:

- Create a new variable, `percent_tip` that computes the percentage tip for each meal
- Summarize the dataset to show the average tip percent by sex and day of week
- Create a line chart using `geom_line()` to show trends over time
 - Hint: Use both the `color` and `group` aesthetic in `aes()`
 - Hint: Use `scale_x_discrete()` so that the days are in increasing order (Thur, Fri, Sat, Sun)
- Based on the graphic you created, what are some observations that you make relating tip percentage to sex and day of week?

```
tips <- read.csv("https://collinn.github.io/data/tips.csv")
```

Problem 9: Intensive care units, or ICUs, are primary spaces in hospitals that are reserved for patients in critical condition. The data below is a random sample of $n = 200$ ICU patients from a research hospital affiliated with Carnegie Mellon University (CMU).

```
icu <- read.csv("https://collinn.github.io/data/icuadmit.csv")
```

Descriptions of the relevant variables are indicated below:

- **ID** - Patient ID number
- **Status** - Patient status: 0=lived or 1=died
- **Age** - Patient's age (in years)
- **Infection** - Is infection involved? 0=no or 1=yes
- **Previous** - Previous admission to ICU within 6 months? 0=no or 1=yes

Using the `icu` dataset and the functions described in this lab, complete the following stages:

1. Change the `Previous` variable to have values "no" and "yes" instead of 0 and 1
2. Filter the data to *only* include patients whose visit involves an infection
3. For the `Age` variable, find the mean, standard deviation, and group size (found using the function `n()`) of patients with and without a previous admission to the ICU in the prior 6 months.

Your solution should indicate the total number of patients with and without previous admission, along with each group's mean age and standard deviation. It will contain two rows and four columns – your final output should look like this:

```
## Previous MeanAge SDAge N
## 1      no  62.369 17.842 65
## 2      yes  57.000 17.601 19
```